

Stepwise Distributed Execution for Load Reduction in Machine Learning Systems

1st Toshihiro Ogino
Shibaura Institute of Technology
Tokyo, Japan
al21066@shibaura-it.ac.jp

2nd Hiroaki Fukuda
Shibaura Institute of Technology
Tokyo, Japan
hiroaki@shibaura-it.ac.jp

3rd Paul Leger
Catholic University of the North
Coquimbo, Chile
pleger@ucn.cl

Abstract—In recent years, machine learning models such as deep neural networks (DNN) and convolutional neural networks (CNN) have been widely adopted across various applications. Image processing, in particular, has seen rapid advancement with the development of high-accuracy models such as the Vision Transformer (ViT). Performing inference using image processing on the edge devices is important from several aspects such as communication latency, preserving privacy and ensuring real-time response. However, in the edge environment where high-performance computing resources such as the cloud cannot be utilized, the inference time tends to be long, causing delays in processing tasks such as anomaly detection using video or image data. To address this issue, we propose a method that reduces its latency applying stepwise distributed executions that consists of simplified multiple CNNs and the optimized ViT. We show its performance through experiments on image classification tasks using edge devices.

Index Terms—Distributed Computing, Edge Computing, Machine Learning, Vision Transformer, CNN

I. INTRODUCTION

The application of machine learning models has become widespread across diverse domains in recent years [1]. Within this context, image processing has emerged as a particularly vibrant area of research, leading to the development of highly accurate classification models like the ViT [2]. However, the inference of high-accuracy machine learning models requires a tremendous amount of computation. Therefore, it is common to utilize external high-performance computing resources, such as those provided by the cloud, to perform inference.

However, due to privacy and security concerns regarding the data being used, it may not be possible to transmit data externally. In such cases, or in environments where internet connectivity infrastructure does not exist—such as in edge computing scenarios—it becomes difficult to perform inference using external computing resources. In these edge computing environments with limited communication capabilities, inference delays may occur due to the lack of using high-performance computers. This, in turn, can hinder processing tasks such as anomaly detection using video or images.

In this paper, we propose a method to reduce the inference load of image classification tasks by performing both training and inference of machine learning models applying stepwise distributed executions without relying on external resources such as cloud computing. We then evaluate the effectiveness of our proposal.

II. RELATED WORK

It is widely known that introducing early-exits in machine learning models can reduce inference time. Surat et al. [3] propose a method called “BranchyNet” for image classification tasks, which adds a branching structure to DNN [4]. This method introduces multiple exit points at intermediate layers of the network, each capable of performing individual inference. If the confidence level of the inference result at a given branch is deemed sufficiently high, the process is terminated at that point, skipping the subsequent layers, thereby reducing the overall computational cost.

Additionally, there have been proposals to reduce inference time and communication cost by splitting the machine learning model into front and back parts, with computation distributed between the client and server [5]. Such approaches, known as Distributed Deep Neural Networks, have gained particular attention in the context of edge computing.

In [3], the confidence (or certainty) of inference results in multi-class classification is evaluated using entropy. Equation 1 represents the output entropy on a normalized scale (0 to 1), based on the inference result $\mathbf{P}$ [5].

$$Entropy = - \sum_{i=1}^C \frac{\mathbf{p}_i \log \mathbf{p}_i}{\log(C)} \quad (1)$$

Here, C represents the number of classification classes, and $\mathbf{p}_i$ denotes the probability of belonging to class i . Entropy serves as an indicator of the dispersion of output probabilities: the closer the distribution is to a one-hot vector, the closer the entropy value approaches 0; the closer it is to a uniform distribution, the higher the entropy value becomes. In [3], if the entropy value obtained from Equation 1 becomes below a certain threshold, the inference is considered sufficiently reliable and is terminated at that point.

Recently, ViT have demonstrated high performance in the field of image recognition and are gaining attention as a powerful alternative to CNN in image classification and object detection tasks [2]. In ViT, the input image is divided into fixed-size patches, which are linearly transformed into vectors. These vectors are then augmented with positional information and input to a *Transformer encoder*. The encoder consists of multiple *Attention Blocks* (composed of Multi-Head Attention

and MLP), which progressively aggregate the input information. The final feature vector obtained is input to a classifier called the MLP Head to produce the inference result. While ViT enables high-accuracy inference, each Attention Block incurs significant computational cost, resulting in a longer overall inference time. Since ViT aggregates features through repeated processing in each block, it is considered feasible to reduce computational cost while maintaining inference accuracy by introducing early-exits at intermediate layers, as in BranchyNet.

III. STEPWISE DISTRIBUTED EXECUTION

Building upon previous works, we propose stepwise distributed execution (we call SDE in this paper) that divides the entire process into multiple stages and distributes its execution across multiple edge devices. The aim is to reduce the computational load on individual edge devices and enhance overall system inference efficiency by simplifying a portion of each inference task. A key advantage of SDE is its ability to perform inference with reduced reliance on high-performance GPU-equipped computers.

For the simplicity of the explanation, we assume only two stages in this section: the first stage and the final stage. SDE utilizes a collaborative approach employing two types of models for inference tasks: a set of lightweight and simplified inference models for the first stage, and a high-accuracy inference model with a higher computational cost for the final stage when needed. The 'inference simplification' here refers to limiting the number of inferred classes in the first stage. For instance, in image classification, the computational cost is reduced as the number of target classes decreased. This feature is effective when the target class possesses specific characteristics that make its inference challenging and/or when a significant number of input images are classified into this particular class compared to others.

Besides, we also apply techniques for reducing the computational cost to the high-accuracy inference model. Following the approach by [3], we apply an Early-Exit mechanism featuring branch structures at intermediate layers in ViT (hereinafter referred to as EE-ViT). This Early-Exit enables providing the output earlier within this step if sufficient inference confidence fulfills the predefined threshold, thereby omitting unnecessary subsequent processing and reducing computation time within that stage.

The SDE generally consists of multiple layers each layer of which also consists of multiple CNNs and the aggregator, which aggregates results from the CNNs as shown in Figure 1. The last aggregator decides and returns the target category when the confidence metrics exceed the predefined threshold (Figure 1-(a)). Otherwise it delegate additional inference to the EE-ViT running on a high-performance computer (Figure 1-(b)). As previously mentioned, EE-ViT may terminate the inference and return the target category in the intermediate layer (Figure 1-(c)). The number of CNNs managed by an aggregator and the number of layers can be arranged based on the tasks to be solved. Moreover, each aggregator could

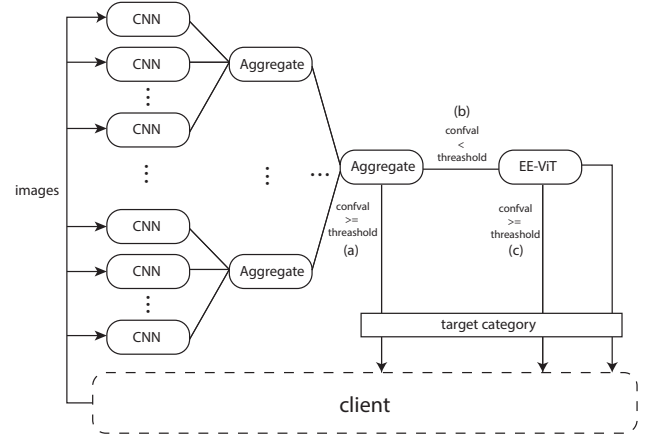


Fig. 1: SDE Architecture

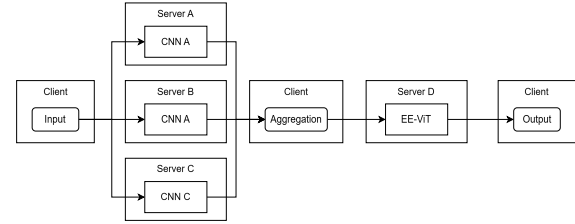


Fig. 2: An architecture of PoC implementation

return the target category to the client (which is not represented in the Figure 1), that can be considered as variants of this architecture. Finally, the confidence metrics is calculated using an entropy-based metric as proposed by [4], and the confidence threshold for triggering the transition between steps is experimentally determined depending on the characteristics of the dataset and the desired performance.

IV. PROOF OF CONCEPT IMPLEMENTATION

In this section, we give a proof of concept implementation for our SDE applied to a multi-class image classification task. In this implementation, we use the CIFAR-10 dataset [6] that is a widely used benchmark dataset composed of color images sized 32×32 pixels, classified into 10 categories. In this implementation, as shown in Figure 2, we use four servers for the classification tasks and one client delegates the classification task and also aggregates results. The four servers consists of three resource limited computers that run lightweight CNNs, and one computer that has sufficient resources and run EE-ViT. The detailed specification will be shown in the next section. The following sections detail the lightweight CNN, EE-ViT and entropy-based confidence metrics as follows.

A. Lightweight CNN

To accelerate inference, we adopt a lightweight CNN with approximately 500,000 parameters. This CNN consists of two

TABLE I: Accuracy of CNN

CNN	Target Category	Accuracy(%)
A	Airplane, Automobile, Bird, Cat	75
B	Deer, Dog, Frog	82
C	Horse, Ship, Truck	86

True label \ Predicted label	Other	Airplane	Automobile	Bird	Cat
Other	5526	174	105	97	98
Airplane	249	718	11	18	4
Automobile	287	20	689	4	0
Bird	514	101	3	356	26
Cat	735	24	7	32	202

(a) CNN AccVsBlocks

True label \ Predicted label	Other	Deer	Dog	Frog
Other	6408	147	316	129
Deer	438	486	25	51
Dog	409	17	563	11
Frog	239	26	18	717

(b) CNN B

True label \ Predicted label	Other	Horse	Ship	Truck
Other	6339	251	224	186
Horse	264	715	2	19
Ship	161	5	809	25
Truck	195	18	55	732

(c) CNN C

Fig. 3: Confusion matrix of CNN

convolutional layers and a fully connected layer. Since its classification performance over the entire CIFAR-10 dataset is limited, we divide the 10 classes of CIFAR-10 into three groups based on classification difficulty and train separate CNNs (CNN A, B, and C) for each group. Table I shows the target classes and accuracy of each CNN. For instance, CNN A classifies five classes: Airplane, Automobile, Bird, Cat, and Other (which includes the remaining classes).

Each CNN was trained on 50,000 training images from CIFAR-10 with data augmentation techniques such as brightness adjustment and affine transformation [7]. Due to differences in classification difficulty across categories, the accuracy of each model varies. Figure 3 shows the confusion matrix of each model. Classes like Cat and Bird exhibit higher misclassification rates, whereas Ship and Truck show relatively high accuracy.

B. Early-Exit Vision Transformer

The structure of the EE-ViT is shown in Figure 4. The input image is divided into 16×16 patches, positional information is added, and the patches are passed through multiple layers of Transformer encoders.

EE-ViT includes several early-exit points at intermediate layers. At each stage, the confidence score is evaluated to

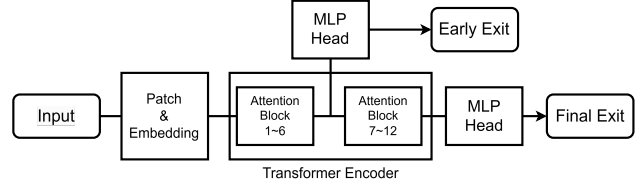


Fig. 4: Structure of Early-Exit ViT

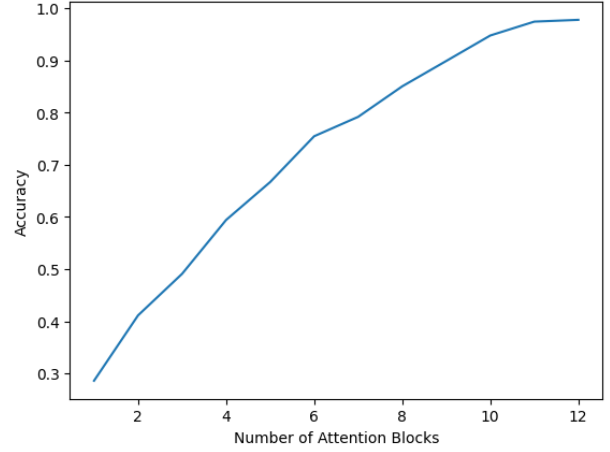


Fig. 5: Relationship between branching point of output and accuracy

decide whether to finalize the output. If the confidence metrics exceeds the predefined threshold, the inference is terminated early, saving computational resources. We use a pre-trained 12-layer ViT model, which is fine-tuned on the CIFAR-10 dataset [8]–[10]. The data augmentation techniques are the same as those used for CNN training. ViT consists of multiple attention blocks that aggregate features at each layer. Figure 5 shows the accuracy at different exit points of EE-ViT. The deeper the early-exit point, the higher the accuracy, suggesting a trade-off between accuracy and computation cost. In this study, the early-exit is placed at the 6th block.

C. Entropy-based Confidence metrics

Following previous studies, we evaluate the reliability of the results from CNNs and the EE-ViT using entropy-based confidence metrics. For obtaining adequate confidence metrics, we conducted the inference using the training dataset and trained model, then investigated the correlation between the inference accuracy and confidence metrics as shown in Figure 6. The white bars indicate correct classifications, while the black bars represent incorrect ones. The results show that incorrect inferences tend to have higher entropy.

For each model, we determine an entropy threshold as confidence metrics that achieves 95% accuracy on the training data and use this to judge whether an inference result can be

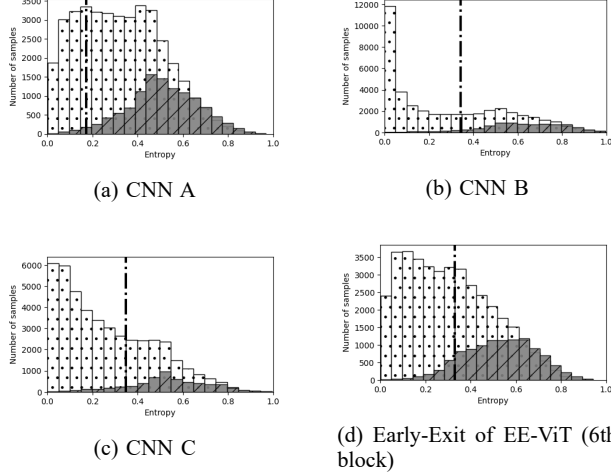


Fig. 6: Entropy distribution of inference results

trusted. We also observed the similar correlation on the test data. There is an inherent trade-off between inference speed and accuracy. Adjusting the threshold offers a mechanism to balance these factors. Lowering this threshold allows for earlier termination of the inference process, thereby reducing computational cost and inference time at the potential expense of accuracy.

V. EVALUATION

We evaluate the effectiveness of SDE using our proof of concept implementation via experiments. In this experiments, we measure two values: Accuracy and Latency for inferences to evaluate SDE. We first describe the experimental procedures and show the results, then discuss the effectiveness throughout the results. Note that, t_x in the procedure represents Time the details of which is explained afterward.

A. Experimental Procedure

We conducted the experiments using four different inference configurations as follows:

- CNN + EE-ViT: Inference configuration combining a lightweight CNNs and a EE-ViT
- CNN + ViT: Combination of CNN and traditional ViT
- EE-ViT only: Inference using only EE-ViT, without CNNs
- ViT only: Inference using traditional ViT

The inference procedure that uses both CNN and (EE-)ViT is as follows:

- 1) The client sends an image to Servers 1, 2, and 3 (t_0).
- 2) Each server performs inference using CNN and returns the result which is a pair of inferred the category and its corresponding confidence metrics to the client.
- 3) The client compares results from the three servers and attempts an early-exit based on confidence thresholds (t_1).

TABLE II: Environment

Role	Computer
Client	RaspBerry Pi 5
Server 1(CNN A)	RaspBerry Pi 5
Server 2(CNN B)	RaspBerry Pi 5
Server 3(CNN C)	RaspBerry Pi 5
Server 4(EE-ViT)	Desktop PC (AMD Ryzen 7 7700X)

- 4) If a sufficiently confident prediction cannot be obtained, the client delegates additional inference to Server 4.
- 5) The inference result from (EE-)ViT is then returned to the client from Server4. Using EE-ViT may return the result early as shown in Figure 4 (t_2).

In the above inference procedure, the starting time is t_0 , and the t_1 and t_2 are the times at which the client receives the inference results for each step and calculates the confidence metrics. The inference times at each point are defined as:

- $t_1 - t_0$: Inference time when early-exit via CNN is successful
- $t_2 - t_0$: Total inference time including EE-ViT

In the cases of using only (EE-)ViT, we simply measure the time based on transmission between the client and Server 4.

We measure latency and the accuracy of each step as an evaluation metric for SDE. These experiments are conducted by dividing 500 images into one batch, with a total of 20 batches measured. The specifications of computers that are used in this experiments are shown in Table II, and all computers are connected via a wired network. The desktop PC utilized for Server 4, incorporating with an AMD Ryzen 7 7700X, represents a more accessible and potentially cost-effective high-performance computing resource compared to specialized data center GPUs and large-scale cloud-based systems. This choice aligns with the core objective of SDE to reduce reliance on external high-performance computing infrastructures while still enabling the execution of computationally intensive models like EE-ViT for accurate inference within a more localized and manageable environment.

B. Experimental Result

Table III shows the inference accuracy for each configuration. In the first place, looking at the CNN + EE-ViT configuration, it can maintain a high accuracy, that is within 1% degradation compared to using only ViT that can considered to show the highest result (97.7%) Furthermore, adjusting the confidence threshold for Early-Exit make it possible to effectively balance the accuracy and its inference latency.

Table IV presents the inference latencies for each configuration. The key observations are as follows: Compared to a traditional ViT (1004s), the CNN + EE-ViT configuration reduced inference time to 794s (approximately 210 seconds reduction), demonstrating the effectiveness of the SDE in this implementation. In addition, the CNN inference stage (executed in parallel on three Rasperry Pi devices) completes in less than 30 seconds. The use of early-exit through CNN contributes to approximately 150 seconds of time saving

TABLE III: Accuracy

Model	CNN(%)	EE-ViT Early (%)	ViT Final (%)	Total(%)
CNN + EE-ViT	95.9	98.0	96.9	96.9
CNN + ViT	95.9	-	97.3	97.1
EE-ViT	-	97.0	96.8	96.9
ViT	-	-	97.7	97.7

TABLE IV: Prediction Time

Model	CNN(Sec)	EE-ViT or ViT(Sec)	Total(Sec)
CNN + EE-ViT	29	765	794
CNN + ViT	28	824	852
EE-ViT	-	855	855
ViT	-	1004	1004

compared to EE-ViT alone, and 210 seconds compared to traditional ViT. These results suggest that SDE is effective in reducing inference cost.

Moreover, Table V shows the percentage of inferences completed at first stages for each configuration. It is notable that the use of Early-Exit reduces the execution rate of the final stage inference by up to 30%. Furthermore, even only using CNNs successfully completes inference via early-exit for 17% of inputs, contributing to overall acceleration.

C. Discussion

While traditional ViT requires 1004 seconds for inference, the SDE reduces this to 794 seconds. This implies that the utilization time of the high-performance Server 4 is reduced by about 240 seconds, or approximately 25%. Inference using machine learning models often results in CPU/GPU utilization exceeding 90%, along with substantial RAM/VRAM usage. Therefore, reducing inference time leads directly to more efficient use of computational resources and improved system response.

The accuracy of early-exits from CNN and EE-ViT tends to be 10 – 20% lower than the best accuracy of ViT (97.7%). The primary reason for CNN's lower accuracy is its lightweight nature, which is the trade-off the accuracy and latency. In addition, variations in accuracy are influenced by differences in classification difficulty among CIFAR-10 categories. For example, categories such as Bird and Cat show a high number of misclassifications (see Figure 3), making early-exit difficult for those cases.

To address this, potential solutions include deploying specialized models for difficult categories or adjusting confidence thresholds per category based on classification difficulty. Such approaches may help achieve both higher early-exit accuracy and shorter inference times, and should be explored in future work.

VI. CONCLUSION

Significant progress in image processing has been propelled by the development of machine learning models, exemplified by the ViT. Performing inference on edge devices is critical for reducing latency, preserving privacy, and achieving real-time

TABLE V: Percentage of Early-Exit Completed

Model	CNN(%)	EE-ViT or ViT (%)	Total(%)
CNN + EE-ViT	17	12	71
CNN + ViT	17	-	83
EE-ViT	-	31	69
ViT	-	-	100

processing. However, due to limited computational resources on the edge, inference often incurs delays, hindering tasks such as anomaly detection in image or video data.

To this end, this paper proposes stepwise distributed execution called SDE combining a lightweight CNN with an EE-ViT. SDE uses a multi-stage inference strategy, starting with rapid, less resource-intensive initial processing, then it escalates to a more precise, resource intensive stage. SDE also terminate inference processes when the confidence metrics exceeds the predefined threshold, reducing subsequent computations. We also give the proof of concept implementation of SDE, then conduct experiments via image classification tasks with CIFAR-10 dataset. Experimental results show the effectiveness of SDE: inference with the CNN + EE-ViT configuration reduces inference time by 20% with less than 1% loss of inference accuracy. Thereby, our proposed SDE presents a promising solution for balancing resource constraints and accuracy requirements, offering high practical value and scalability, particularly for applications in the edge computing and AI domain.

As for future research, we plan to apply SDE to other tasks such as regressions finding an adequate confidence metrics that is essential for terminating in the earlier stages.

REFERENCES

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [3] S. Scardapane, M. Scarpiniti, E. Baccarelli, and A. Uncini, "Why should we add early exits to neural networks?" *Cognitive Computation*, vol. 12, no. 5, p. 954 – 966, Jun. 2020. [Online]. Available: <http://dx.doi.org/10.1007/s12559-020-09734-4>
- [4] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," 2017. [Online]. Available: <https://arxiv.org/abs/1709.01686>
- [5] —, "Distributed deep neural networks over the cloud, the edge and end devices," in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 328–339.
- [6] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.
- [7] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [8] R. Wightman, "Pytorch image models," <https://github.com/rwightman/pytorch-image-models>, 2019.
- [9] I. Z. Yalniz, H. Jégou, K. Chen, M. Paluri, and D. Mahajan, "Billion-scale semi-supervised learning for image classification," 2019.
- [10] D. K. Mahajan, R. B. Girshick, V. Ramanathan, K. He, M. Paluri, Y. Li, A. Bharambe, and L. van der Maaten, "Exploring the limits of weakly supervised pretraining," in *ECCV*, 2018.